

Lecture 01 - Introduction

This lecture is the orientation layer for the whole course. It explains why interactive computer graphics is a pipeline problem: a scene must be represented, processed under time constraints, sampled into an image, and displayed fast enough for interaction.

Source chunk	course_text_parts/03_lectures/01_introduction.txt
Extracted slide pages	51
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Course Learning Outcomes

Source cue	- Learn to program computer graphics applications using OpenGL / - Learn to realize efficient computer graphics applications on the GPU / - Implement your own computer graphics project / [Image Credits: Wikipedia User Mimigu] [Crassin et al., PG2011]
Professor-style explanation	The slide 'Course Learning Outcomes' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: - Learn to program computer graphics applications using OpenGL / - Learn to realize efficient computer graphics applications on the GPU / - Implement your own computer graphics project / [Image Credits: Wikipedia User Mimigu] [Crassin et al., PG2011]. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about Course Learning Outcomes. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: - Learn to program computer graphics applications using OpenGL / - Learn to realize efficient computer graphics applications on the GPU / - Implement your own computer graphics project / [Image Credits: Wikipedia User Mimigu] [Crassin et al., PG2011]. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Course Learning Outcomes' to an exercise, project task, or exam-preparation action?
Page 3	Real-Time Rendering Contest Results 1/3
Source cue	Students: Frank Richter, Manuel Herbert Güntzel

Professor-style explanation	The slide 'Real-Time Rendering Contest Results 1/3' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: Students: Frank Richter, Manuel Herbert Güntzel. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Real-Time Rendering Contest Results 1/3. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: Students: Frank Richter, Manuel Herbert Güntzel. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Real-Time Rendering Contest Results 1/3' and why it mattered historically?
Page 4	Real-Time Rendering Contest Results 2/3
Source cue	Students: Andreas Rottach, Matthias Englert
Professor-style explanation	The slide 'Real-Time Rendering Contest Results 2/3' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: Students: Andreas Rottach, Matthias Englert. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.

Technical commentary	This slide is about Real-Time Rendering Contest Results 2/3. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: Students: Andreas Rottach, Matthias Englert. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Real-Time Rendering Contest Results 2/3' and why it mattered historically?

Page 5	Real-Time Rendering Contest Results 3/3
Source cue	Students: Jan Eric Haßler, Kai Viktor Freissler
Professor-style explanation	The slide 'Real-Time Rendering Contest Results 3/3' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: Students: Jan Eric Haßler, Kai Viktor Freissler. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Real-Time Rendering Contest Results 3/3. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: Students: Jan Eric Haßler, Kai Viktor Freissler. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Real-Time Rendering Contest Results 3/3' and why it mattered historically?
Page 6	1.1 Course Organization
Source cue	Details about the lecture course

Professor-style explanation	The slide '1.1 Course Organization' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: Details about the lecture course. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about 1.1 Course Organization. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: Details about the lecture course. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect '1.1 Course Organization' to an exercise, project task, or exam-preparation action?
Page 7	Teachers
Source cue	- Lectures / timo.ropinski@uni-ulm.de / Office hours upon request / - Exercises / Poonam Aditya Sawant
Professor-style explanation	The slide 'Teachers' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: - Lectures / timo.ropinski@uni-ulm.de / Office hours upon request / - Exercises / Poonam Aditya Sawant. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.

Technical commentary	This slide is about Teachers. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: - Lectures / timo.ropinski@uni-ulm.de / Office hours upon request / - Exercises / Poonam Aditya Sawant. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Teachers' to an exercise, project task, or exam-preparation action?

Page 8	Course Format
Source cue	- Lectures / - Tuesday (14-16) and every other Wednesdays (12-14) / - Room O28/1002 / - Specific dates/modalities in Moodle (password: ICG26SUMMER) / - Lecture slides
Professor-style explanation	The slide 'Course Format' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: - Lectures / - Tuesday (14-16) and every other Wednesdays (12-14) / - Room O28/1002 / - Specific dates/modalities in Moodle (password: ICG26SUMMER) / - Lecture slides. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about Course Format. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: - Lectures / - Tuesday (14-16) and every other Wednesdays (12-14) / - Room O28/1002 / - Specific dates/modalities in Moodle (password: ICG26SUMMER) / - Lecture slides. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Course Format' to an exercise, project task, or exam-preparation action?

Page 9	Course Format
--------	---------------

Source cue	- Exercises / - Every other Wednesday (12-14) / - Room O28/1002 / - Programming in C/C++ (introduction available upon request) / - Enroll in Moodle
Professor-style explanation	The slide 'Course Format' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: - Exercises / - Every other Wednesday (12-14) / - Room O28/1002 / - Programming in C/C++ (introduction available upon request) / - Enroll in Moodle. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about Course Format. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: - Exercises / - Every other Wednesday (12-14) / - Room O28/1002 / - Programming in C/C++ (introduction available upon request) / - Enroll in Moodle. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Course Format' to an exercise, project task, or exam-preparation action?
Page 10	Exercises
Source cue	- Exercise / - Exercise 1 - Rendering Pipeline / - Exercise 2 - Primitive Types and Shaders / - Exercise 3 - Geometric Transformations / - Exercise 4 - Projection and Clipping

Professor-style explanation	The slide 'Exercises' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: - Exercise / - Exercise 1 - Rendering Pipeline / - Exercise 2 - Primitive Types and Shaders / - Exercise 3 - Geometric Transformations / - Exercise 4 - Projection and Clipping. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about Exercises. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: - Exercise / - Exercise 1 - Rendering Pipeline / - Exercise 2 - Primitive Types and Shaders / - Exercise 3 - Geometric Transformations / - Exercise 4 - Projection and Clipping. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Exercises' to an exercise, project task, or exam-preparation action?

Page 11	Successful Course Completion
Source cue	- Pass the oral/written exam / - Dates to be announced / - "Notenbonus" possible / - 50% success rate of exercise solutions for n-1 sheets / - Only running solutions are marked
Professor-style explanation	The slide 'Successful Course Completion' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: - Pass the oral/written exam / - Dates to be announced / - "Notenbonus" possible / - 50% success rate of exercise solutions for n-1 sheets / - Only running solutions are marked. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.

Technical commentary	This slide is about Successful Course Completion. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: - Pass the oral/written exam / - Dates to be announced / - "Notenbonus" possible / - 50% success rate of exercise solutions for n-1 sheets / - Only running solutions are marked. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Successful Course Completion' to an exercise, project task, or exam-preparation action?

Page 12	Course Overview
Source cue	Background / Journey along the / Rendering Pipeline / Lighting Effects / Extrahierte Tabellen:
Professor-style explanation	The slide 'Course Overview' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: Background / Journey along the / Rendering Pipeline / Lighting Effects / Extrahierte Tabellen. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about Course Overview. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: Background / Journey along the / Rendering Pipeline / Lighting Effects / Extrahierte Tabellen. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Course Overview' to an exercise, project task, or exam-preparation action?

Page 13	Literature
Source cue	- P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics / (4th Edition), AK Peters 2016. / - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013.

Professor-style explanation	The slide 'Literature' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. The listed sources support the chapter content: - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics / (4th Edition), AK Peters 2016. / - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide.
Technical commentary	This slide is about Literature. The slide lists source material or chapter references that support the technical content and give names for further reading. The listed sources support the chapter content: - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics / (4th Edition), AK Peters 2016. / - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic 'Literature' points to for deeper study?

Page 14	Relevant Chapters and Link to Lecture Course
Source cue	- For self study
Professor-style explanation	The slide 'Relevant Chapters and Link to Lecture Course' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: - For self study. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about Relevant Chapters and Link to Lecture Course. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: - For self study. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.

Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Relevant Chapters and Link to Lecture Course' to an exercise, project task, or exam-preparation action?
Page 15	(1.1 Course Organization)
Source cue	1.2 Computer Graphics Overview / 1.3 Pixel-based Representations / 1.4 3D Models / 1.5 Algorithmic Paradigms
Professor-style explanation	The slide '(1.1 Course Organization)' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: 1.2 Computer Graphics Overview / 1.3 Pixel-based Representations / 1.4 3D Models / 1.5 Algorithmic Paradigms. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about (1.1 Course Organization). The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: 1.2 Computer Graphics Overview / 1.3 Pixel-based Representations / 1.4 3D Models / 1.5 Algorithmic Paradigms. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect '(1.1 Course Organization)' to an exercise, project task, or exam-preparation action?
Page 16	1.2 Computer Graphics Overview
Source cue	Relation of the field with respect to other disciplines
Professor-style explanation	The slide '1.2 Computer Graphics Overview' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: Relation of the field with respect to other disciplines. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about 1.2 Computer Graphics Overview. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: Relation of the field with respect to other disciplines. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn '1.2 Computer Graphics Overview' into a causal sentence instead of repeating the slide title?
Page 17	Computer Graphics History
Source cue	- Origins in art / - Perspective, color, etc. / - Computer graphics milestones [Reichardt, 1968] / - 1960 - "Computer Graphics", William Fetter (Boeing) / - 1962 - Spacewars, Steve Russel (MIT) Computer
Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: - Origins in art / - Perspective, color, etc. / - Computer graphics milestones [Reichardt, 1968] / - 1960 - "Computer Graphics", William Fetter (Boeing) / - 1962 - Spacewars, Steve Russel (MIT) Computer. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: - Origins in art / - Perspective, color, etc. / - Computer graphics milestones [Reichardt, 1968] / - 1960 - "Computer Graphics", William Fetter (Boeing) / - 1962 - Spacewars, Steve Russel (MIT) Computer. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?
Page 18	Computer Graphics History
Source cue	Sutherland, Utah (1970) Wireframe

Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: Sutherland, Utah (1970) Wireframe. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: Sutherland, Utah (1970) Wireframe. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?

Page 19	Computer Graphics History
Source cue	Sutherland, Utah (1970) Turner Whitted, Siggraph (1979)
Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: Sutherland, Utah (1970) Turner Whitted, Siggraph (1979). Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: Sutherland, Utah (1970) Turner Whitted, Siggraph (1979). Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.

Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?
Page 20	Computer Graphics History
Source cue	The Abyss (1989) Terminator 2 (1991) Jurassic Park (1993)
Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: The Abyss (1989) Terminator 2 (1991) Jurassic Park (1993). Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: The Abyss (1989) Terminator 2 (1991) Jurassic Park (1993). Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?
Page 21	Computer Graphics History
Source cue	Ikea (2014)

Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: Ikea (2014). Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: Ikea (2014). Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?

Page 22	Related Disciplines
Source cue	- Human-Computer Interaction / - Virtual and Augmented Reality / - Visualization / - Image Processing / - Computer Vision
Professor-style explanation	The slide 'Related Disciplines' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Human-Computer Interaction / - Virtual and Augmented Reality / - Visualization / - Image Processing / - Computer Vision. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Related Disciplines. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Human-Computer Interaction / - Virtual and Augmented Reality / - Visualization / - Image Processing / - Computer Vision. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Related Disciplines' into a causal sentence instead of repeating the slide title?
Page 23	Applications
Source cue	- Computer games / - Animation movies / - Special effects in movies / - CAD/CAM [CryEngine 5.7, CryTek 2022] / - Simulation
Professor-style explanation	The slide 'Applications' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Computer games / - Animation movies / - Special effects in movies / - CAD/CAM [CryEngine 5.7, CryTek 2022] / - Simulation. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Applications. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Computer games / - Animation movies / - Special effects in movies / - CAD/CAM [CryEngine 5.7, CryTek 2022] / - Simulation. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Applications' into a causal sentence instead of repeating the slide title?
Page 24	1.3 Pixel-based Representations
Source cue	What is a pixel?
Professor-style explanation	The slide '1.3 Pixel-based Representations' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. The numerical values belong to one memory calculation: What is a pixel?. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations.
Technical commentary	This slide is about 1.3 Pixel-based Representations. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. The numerical values belong to one memory calculation: What is a pixel?. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost.

Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in '1.3 Pixel-based Representations'?
Page 25	Raster Images
Source cue	["Bibliothek" by press office Ulm University]
Professor-style explanation	The slide 'Raster Images' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. The numerical values belong to one memory calculation: ["Bibliothek" by press office Ulm University]. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations.
Technical commentary	This slide is about Raster Images. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. The numerical values belong to one memory calculation: ["Bibliothek" by press office Ulm University]. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost.
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in 'Raster Images'?
Page 26	Pixel
Source cue	- Name derived from Word for Picture Element / - Pixel is a sample with associated sample values (=pixel components) / - Color / - Transparency (=alpha value) / - Depth (i.e., distance from viewer (=z value))
Professor-style explanation	The slide 'Pixel' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Name derived from Word for Picture Element / - Pixel is a sample with associated sample values (=pixel components) / - Color / - Transparency (=alpha value) / - Depth (i.e., distance from viewer (=z value)). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.

Technical commentary	This slide is about Pixel. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Name derived from Word for Picture Element / - Pixel is a sample with associated sample values (=pixel components) / - Color / - Transparency (=alpha value) / - Depth (i.e., distance from viewer (=z value)). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Pixel' works per object, per image region, per ray, or per fragment?

Page 27	Color
Source cue	- In real world color is given / by color spectrum Day Light Light Bulb / Fluorescent Tube White LED / - In computer graphics color is represented through color models / - RGB color model (additive)
Professor-style explanation	The slide 'Color' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - In real world color is given / by color spectrum Day Light Light Bulb / Fluorescent Tube White LED / - In computer graphics color is represented through color models / - RGB color model (additive). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Color. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - In real world color is given / by color spectrum Day Light Light Bulb / Fluorescent Tube White LED / - In computer graphics color is represented through color models / - RGB color model (additive). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Color'?

Page 28	LCD Raster Screen
Source cue	- Liquid crystal displays exist since the 90s and enable flat screens / - Liquid crystals affect the polarization of light / - Based on polarization, subpixel visibility is regulated

Professor-style explanation	The slide 'LCD Raster Screen' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Liquid crystal displays exist since the 90s and enable flat screens / - Liquid crystals affect the polarization of light / - Based on polarization, subpixel visibility is regulated. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about LCD Raster Screen. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Liquid crystal displays exist since the 90s and enable flat screens / - Liquid crystals affect the polarization of light / - Based on polarization, subpixel visibility is regulated. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'LCD Raster Screen'?
Page 29	Raster Image Sizes
Source cue	- Assume a raster image with 1024x768 pixels and 24-bit color depth / - How many different colors can be represented? / - $2^{24} = 2^8 \times 2^8 \times 2^8 = 16.777.216$ / - How much memory does the image need? / - $1024 \times 768 \times 24 = 18,874,368 \text{ bits} = 2,359,296 \text{ bytes} = 2.25 \text{ MiB}$
Professor-style explanation	The slide 'Raster Image Sizes' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. The numerical values belong to one memory calculation: - Assume a raster image with 1024x768 pixels and 24-bit color depth / - How many different colors can be represented? / - $2^{24} = 2^8 \times 2^8 \times 2^8 = 16.777.216$ / - How much memory does the image need? / - $1024 \times 768 \times 24 = 18,874,368 \text{ bits} = 2,359,296 \text{ bytes} = 2.25 \text{ MiB}$. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations.

Technical commentary	This slide is about Raster Image Sizes. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. The numerical values belong to one memory calculation: - Assume a raster image with 1024x768 pixels and 24-bit color depth / - How many different colors can be represented? / - $2^{24} = 2^8 \times 2^8 \times 2^8 = 16.777.216$ / - How much memory does the image need? / - $1024 \times 768 \times 24 = 18,874,368$ bits = 2,359,296 bytes = 2.25 MiB. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost.
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in 'Raster Image Sizes'?

Page 30	1.4 3D Models
Source cue	Modeling with triangles and other geometric primitives
Professor-style explanation	The slide '1.4 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: Modeling with triangles and other geometric primitives. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about 1.4 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: Modeling with triangles and other geometric primitives. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by '1.4 3D Models' before rendering begins?

Page 31	3D Model Types
Source cue	- Models specify the structures to be displayed / - Three commonly used model types exist / - Implicit surfaces / - Polygonal models / - Volumetric models

Professor-style explanation	The slide '3D Model Types' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Models specify the structures to be displayed / - Three commonly used model types exist / - Implicit surfaces / - Polygonal models / - Volumetric models. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about 3D Model Types. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Models specify the structures to be displayed / - Three commonly used model types exist / - Implicit surfaces / - Polygonal models / - Volumetric models. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by '3D Model Types' before rendering begins?
Page 32	Implicit Surfaces
Source cue	- Set of all points that fulfill $f(x, y, z) = 0$ / - All points with $f(x, y, z) < 0$ define solid bound by $f(x, y, z) = 0$ / - Display possibilities / - Search for null points / - Approximate with polygons
Professor-style explanation	The slide 'Implicit Surfaces' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Set of all points that fulfill $f(x, y, z) = 0$ / - All points with $f(x, y, z) < 0$ define solid bound by $f(x, y, z) = 0$ / - Display possibilities / - Search for null points / - Approximate with polygons. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Implicit Surfaces. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Set of all points that fulfill $f(x, y, z) = 0$ / - All points with $f(x, y, z) < 0$ define solid bound by $f(x, y, z) = 0$ / - Display possibilities / - Search for null points / - Approximate with polygons. The slide groups them because one object or quantity is processed into another result used later in rendering.

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Implicit Surfaces' into a causal sentence instead of repeating the slide title?
Page 33	Quadrics
Source cue	- Surfaces represented by quadric equations / Ellipsoid Single Hyperboloid Double Hyperboloid / $x^2 y^2 z^2$ / $f(x, y, z) = + - 1 f(x, y, z) = + - - 1 f(x, y, z) = + - + 1 / \alpha^2 \beta^2 \gamma^2$
Professor-style explanation	The slide 'Quadrics' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Surfaces represented by quadric equations / Ellipsoid Single Hyperboloid Double Hyperboloid / $x^2 y^2 z^2$ / $f(x, y, z) = + - 1 f(x, y, z) = + - - 1 f(x, y, z) = + - + 1 / \alpha^2 \beta^2 \gamma^2$. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Quadrics. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Surfaces represented by quadric equations / Ellipsoid Single Hyperboloid Double Hyperboloid / $x^2 y^2 z^2$ / $f(x, y, z) = + - 1 f(x, y, z) = + - - 1 f(x, y, z) = + - + 1 / \alpha^2 \beta^2 \gamma^2$. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Quadrics' into a causal sentence instead of repeating the slide title?
Page 34	Object Representation
Source cue	Point Cloud (3D Scan) Binary Data (MRT) Signed Distance Fields (SDF) Meshes
Professor-style explanation	The slide 'Object Representation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: Point Cloud (3D Scan) Binary Data (MRT) Signed Distance Fields (SDF) Meshes. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about Object Representation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: Point Cloud (3D Scan) Binary Data (MRT) Signed Distance Fields (SDF) Meshes. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Object Representation' into a causal sentence instead of repeating the slide title?
Page 35	Polygonal Models 1/3
Source cue	- Surface representation is approximated by polygons / - Polygons are made up of vertices and edges
Professor-style explanation	The slide 'Polygonal Models 1/3' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Surface representation is approximated by polygons / - Polygons are made up of vertices and edges. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Polygonal Models 1/3. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Surface representation is approximated by polygons / - Polygons are made up of vertices and edges. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 1/3' before rendering begins?
Page 36	Polygonal Models 1/4
Source cue	- Surface representation is approximated by polygons (mostly triangles/quads) / - Polygons are made up of vertices and edges / Valence / Degree Adjacent Edges 1-Ring Neighborhood

Professor-style explanation	The slide 'Polygonal Models 1/4' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Surface representation is approximated by polygons (mostly triangles/quads) / - Polygons are made up of vertices and edges / Valence / Degree Adjacent Edges 1-Ring Neighborhood. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Polygonal Models 1/4. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Surface representation is approximated by polygons (mostly triangles/quads) / - Polygons are made up of vertices and edges / Valence / Degree Adjacent Edges 1-Ring Neighborhood. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 1/4' before rendering begins?
Page 37	Polygonal Models 2/4
Source cue	- Surface representation are often approximated by triangular polygons / - Triangles consist of three vertices and three edges / - Display achieved through rendering by raster conversion of triangles
Professor-style explanation	The slide 'Polygonal Models 2/4' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Surface representation are often approximated by triangular polygons / - Triangles consist of three vertices and three edges / - Display achieved through rendering by raster conversion of triangles. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.

Technical commentary	This slide is about Polygonal Models 2/4. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Surface representation are often approximated by triangular polygons / - Triangles consist of three vertices and three edges / - Display achieved through rendering by raster conversion of triangles. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 2/4' before rendering begins?

Page 38	Polygonal Models 3/4
Source cue	- Various attributes are associated with vertices of a polygonal model / - Colors / - Material properties / - Textures / - ...
Professor-style explanation	The slide 'Polygonal Models 3/4' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Various attributes are associated with vertices of a polygonal model / - Colors / - Material properties / - Textures / - . Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Polygonal Models 3/4. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Various attributes are associated with vertices of a polygonal model / - Colors / - Material properties / - Textures / - . Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 3/4' before rendering begins?

Page 39	Polygonal Models 4/4
Source cue	- Surface normal required in many cases / - Normal is calculated through cross product: / - Example use cases / $a \times b = \begin{vmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix}$

Professor-style explanation	The slide 'Polygonal Models 4/4' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Surface normal required in many cases / - Normal is calculated through cross product: $\mathbf{a} \times \mathbf{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix}$ 2. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Polygonal Models 4/4. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Surface normal required in many cases / - Normal is calculated through cross product: $\mathbf{a} \times \mathbf{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix}$ 2. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 4/4' before rendering begins?
Page 40	Volumetric Models
Source cue	- Space filling object representation through voxels (volume elements) / - Display possibilities / - Rendering of extracted surfaces / - Direct rendering through specialized algorithms
Professor-style explanation	The slide 'Volumetric Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Space filling object representation through voxels (volume elements) / - Display possibilities / - Rendering of extracted surfaces / - Direct rendering through specialized algorithms. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.

Technical commentary	This slide is about Volumetric Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Space filling object representation through voxels (volume elements) / - Display possibilities / - Rendering of extracted surfaces / - Direct rendering through specialized algorithms. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Volumetric Models' before rendering begins?

Page 41	Model Generation
Source cue	- Polygonal models are generated through 3D scanning or 3D modeling / (e.g., LIDAR, Blender, Cinema 4D, Maya) / - Volumetric models are generated through imaging or simulation / (e.g., CT, MRI, PET, CFD, seismic surveys)
Professor-style explanation	The slide 'Model Generation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Polygonal models are generated through 3D scanning or 3D modeling / (e.g., LIDAR, Blender, Cinema 4D, Maya) / - Volumetric models are generated through imaging or simulation / (e.g., CT, MRI, PET, CFD, seismic surveys). The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Model Generation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Polygonal models are generated through 3D scanning or 3D modeling / (e.g., LIDAR, Blender, Cinema 4D, Maya) / - Volumetric models are generated through imaging or simulation / (e.g., CT, MRI, PET, CFD, seismic surveys). The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Model Generation' into a causal sentence instead of repeating the slide title?

Page 42	Coordinate Systems
Source cue	- Three main coordinate system exist / - Model coordinate system - model specified in own coordinate system / - World coordinate system - models arranged in common coordinate system / - Eye/camera coordinate system - model in relation to camera / - To transfer between coordinate systems,

Professor-style explanation	The slide 'Coordinate Systems' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Three main coordinate system exist - Model coordinate system - model specified in own coordinate system - World coordinate system - models arranged in common coordinate system - Eye/camera coordinate system - model in relation to camera To transfer between coordinate systems,. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Coordinate Systems. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Three main coordinate system exist - Model coordinate system - model specified in own coordinate system - World coordinate system - models arranged in common coordinate system - Eye/camera coordinate system - model in relation to camera To transfer between coordinate systems,. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Coordinate Systems'?
Page 43	Rendering
Source cue	- Rendering is the process to generate raster images from 3D models - During rendering several steps are necessary - Coordinate system transformation - Visibility determination - Illumination

Professor-style explanation	The slide 'Rendering' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Rendering is the process to generate raster images from 3D models / - During rendering several steps are necessary / - Coordinate system transformation / - Visibility determination / - Illumination. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Rendering. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Rendering is the process to generate raster images from 3D models / - During rendering several steps are necessary / - Coordinate system transformation / - Visibility determination / - Illumination. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Rendering'?
Page 44	1.5 Algorithmic Paradigms
Source cue	Frequently found algorithm strategies
Professor-style explanation	The slide '1.5 Algorithmic Paradigms' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: Frequently found algorithm strategies. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about 1.5 Algorithmic Paradigms. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: Frequently found algorithm strategies. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn '1.5 Algorithmic Paradigms' into a causal sentence instead of repeating the slide title?
Page 45	Photorealistic Rendering
Source cue	- Goal: simulate reality / [Embree Ray Tracing Website - http://embree.github.io/] [Maisch & Ropinski, Eurographics 2017] [Hendrik vann Jenssen]
Professor-style explanation	The slide 'Photorealistic Rendering' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Goal: simulate reality / [Embree Ray Tracing Website - http://embree.github.io/] [Maisch & Ropinski, Eurographics 2017] [Hendrik vann Jenssen]. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Photorealistic Rendering. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Goal: simulate reality / [Embree Ray Tracing Website - http://embree.github.io/] [Maisch & Ropinski, Eurographics 2017] [Hendrik vann Jenssen]. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Photorealistic Rendering' works per object, per image region, per ray, or per fragment?
Page 46	Non-Photorealistic Rendering (NPR)
Source cue	- Goal: abstract reality / [Suggestive contour line drawings, DeCarlo 2003]

Professor-style explanation	The slide 'Non-Photorealistic Rendering (NPR)' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Goal: abstract reality / [Suggestive contour line drawings, DeCarlo 2003]. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Non-Photorealistic Rendering (NPR). The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Goal: abstract reality / [Suggestive contour line drawings, DeCarlo 2003]. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Non-Photorealistic Rendering (NPR)'?

Page 47	Image-Based Algorithms
Source cue	- Number of computations direct proportional to number of pixels / for all Pixels do { / for all SceneObjects do { / ... / - Benefits
Professor-style explanation	The slide 'Image-Based Algorithms' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Number of computations direct proportional to number of pixels / for all Pixels do { / for all SceneObjects do { / ... / - Benefits. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Image-Based Algorithms. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Number of computations direct proportional to number of pixels / for all Pixels do { / for all SceneObjects do { / ... / - Benefits. The slide groups them because one object or quantity is processed into another result used later in rendering.

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Image-Based Algorithms' into a causal sentence instead of repeating the slide title?
Page 48	Object-Based Algorithms
Source cue	- Number of computations direct proportional to number of 3D models / for all SceneObjects do { / for all Pixels do { / ... / - Benefits
Professor-style explanation	The slide 'Object-Based Algorithms' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Number of computations direct proportional to number of 3D models / for all SceneObjects do { / for all Pixels do { / ... / - Benefits. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Object-Based Algorithms. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Number of computations direct proportional to number of 3D models / for all SceneObjects do { / for all Pixels do { / ... / - Benefits. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Object-Based Algorithms' into a causal sentence instead of repeating the slide title?
Page 49	are assigned
Source cue	- Raster images are formed of pixels to which colors, transparencies and depth / are assigned / polygonal models / - Most frequent building blocks of polygonal models are triangles, / which are formed of vertices and edges

Professor-style explanation	The slide 'are assigned' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Raster images are formed of pixels to which colors, transparencies and depth / are assigned / polygonal models / - Most frequent building blocks of polygonal models are triangles, / which are formed of vertices and edges. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about are assigned. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Raster images are formed of pixels to which colors, transparencies and depth / are assigned / polygonal models / - Most frequent building blocks of polygonal models are triangles, / which are formed of vertices and edges. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'are assigned'?

Page 50	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.

Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?
Page 51	Peters 2016. (Chapters 1, 3 & 21)
Source cue	- Text Books / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics (4th Edition), AK / Peters 2016. (Chapters 1, 3 & 21) / - Crassin, Cyril, Fabrice Neyret, Miguel Sainz, Simon Green, and Elmar Eisemann. "Interactive / indirect illumination using voxel cone tracing." In Computer Graphics Forum, vol. 30, no. 7, pp.
Professor-style explanation	The slide 'Peters 2016. (Chapters 1, 3 & 21)' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Text Books / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics (4th Edition), AK / Peters 2016. (Chapters 1, 3 & 21) / - Crassin, Cyril, Fabrice Neyret, Miguel Sainz, Simon Green, and Elmar Eisemann. "Interactive / indirect illumination using voxel cone tracing." In Computer Graphics Forum, vol. 30, no. 7, pp. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Peters 2016. (Chapters 1, 3 & 21). The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Text Books / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics (4th Edition), AK / Peters 2016. (Chapters 1, 3 & 21) / - Crassin, Cyril, Fabrice Neyret, Miguel Sainz, Simon Green, and Elmar Eisemann. "Interactive / indirect illumination using voxel cone tracing." In Computer Graphics Forum, vol. 30, no. 7, pp. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Peters 2016. (Chapters 1, 3 & 21)'?

01.1 Course Organization

Source location: Slides around 6-15

Commentary

The organizational slides matter because they explain how the theory and programming parts fit together. The exercises are not separate from the lecture; they are the practical version of the same pipeline ideas. When the course says that slides are generally not self-explanatory, that is a warning: the bullet points are anchors, not a textbook.

Treat every exercise sheet as an applied checkpoint. Rendering pipeline, primitive types, shaders, transformations, projection, clipping, and rasterization are introduced in the lectures and then turned into OpenGL tasks. If a topic appears both in a lecture and an exercise, it is more likely to be exam relevant.

Mental model

Lecture slides give the vocabulary; exercises force you to connect that vocabulary to code and visible output.

Check yourself

Can you list the five exercise themes and connect each one to a later lecture chapter?

01.2 Computer Graphics Overview

Source location: Section 1.2

Commentary

Computer graphics is about generating images from descriptions. The description can be pixel-based, object-based, physically motivated, artistic, or algorithmic. The common thread is that the computer must decide what color belongs at image samples.

Interactive graphics adds a time constraint. The image is not computed once and admired; it must respond to camera motion, user input, animation, and changing state. That is why real-time rendering often uses approximations and specialized GPU stages.

Mental model

Graphics is controlled image generation. Interactive graphics is controlled image generation under a strict time budget.

Check yourself

What changes when a renderer must be interactive rather than offline?

01.3 Pixel-Based Representations

Source location: Section 1.3

Commentary

A pixel-based representation stores an image directly as samples on a grid. It is easy to display and edit locally, but it does not know the underlying 3D scene. If you zoom into a raster image, you reveal the sampling grid, not more geometry.

This is the first place where sampling becomes important. Resolution, color depth, aliasing, and frame time are not cosmetic details. They decide what information can be represented and how much data must be processed per second.

Mental model

A raster image is already the answer. It contains color samples, not the geometric reasons behind them.

Check yourself

Why can a pixel image be easy to display but hard to reinterpret as a 3D scene?

01.4 3D Models

Source location: Section 1.4

Commentary

A 3D model is a structured description of objects before they become pixels. It may contain vertices, primitives, topology, normals, materials, textures, and transformations. Unlike a pixel image, it can be viewed from different camera positions.

The course mostly follows object-based rendering: start from model data, transform it, project it, rasterize it, shade it, test visibility, and finally update pixels. Later chapters explain these steps one by one.

Mental model

A model is not an image. It is a cause from which many possible images can be generated.

Check yourself

Which model attributes can influence final color besides vertex positions?

01.5 Algorithmic Paradigms

Source location: Section 1.5

Commentary

The lecture contrasts ways to generate images. Rasterization works from objects toward pixels and is the dominant real-time pipeline. Ray-based methods work from image samples into the scene and are powerful for visibility and lighting but can be more expensive.

Do not memorize these paradigms as names only. Ask what direction information flows. Does the algorithm start from geometry and find covered pixels, or from pixels/rays and find visible geometry? That direction explains the strengths and weaknesses.

Mental model

Rasterization asks, 'Which samples does this primitive cover?' Ray casting asks, 'What does this sample see?'

Check yourself

Which paradigm fits OpenGL's standard real-time pipeline most directly?

End-of-lecture summary

If you remember only one thing from Lecture 01, remember this: This lecture is the orientation layer for the whole course. It explains why interactive computer graphics is a pipeline problem: a scene must be represented, processed under time constraints, sampled into an image, and displayed fast enough for interaction.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.